

Patient segmentation and hospital outcomes: adults aged 45-64

Vivienne Gunadhi | Synthetic EHR, 2018-2024

Decision question

Among hospitalized adults aged 45-64, which baseline patient profiles are associated with different readmission, resource-use, and mortality patterns, and which segments merit further investigation for care-coordination planning?

This is an exploratory analysis of **synthetic, observational data**, not an intervention evaluation, clinical prediction system, or estimate of real-world treatment effects. Hospital charges are not costs or profit.

Analysis plan

1. Validate and join patient and outcome tables; define the age cohort upfront.
2. Describe disease burden and hospital outcomes within that cohort.
3. Retain focused observational comparisons and quantify uncertainty.
4. Reduce baseline clinical features with PCA; compare K-means solutions.
5. Profile clusters using outcomes excluded from model fitting.

The full patient population is used only for data-quality checks, cohort counts and two context plots. Adults 65+ appear only as a contextual comparison for charges.

1. Setup and input files

This notebook was run in Google Colab with the CSVs on Google Drive. Update the paths in the next cells if your files live elsewhere; outside Colab, skip the Drive-mount cell.

All saved outputs come from a complete top-to-bottom run in Colab. To reproduce them, use **Restart & Run All** with the source CSVs available.

In [1]:

```
from pathlib import Path
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

SEED = 3330
N_BOOT = 2000
AGE_MIN, AGE_MAX = 45, 64
DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)
sns.set_theme(style="whitegrid", palette=["#5275ad", "#8caadc", "#284570"])
pd.set_option("display.max_columns", 40)
```

In [2]:

```
try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Not running in Colab; skipping Drive mount.")
```

Mounted at /content/drive

In [3]:

```
# Edit DATA_DIR below if your files move.
DATA_DIR = Path("/content/drive/MyDrive/DSO_545/Final")
paths = [DATA_DIR / "patients.csv", DATA_DIR / "patient_outcome.csv"]
missing = [str(p) for p in paths if not p.exists()]
if missing:
    raise FileNotFoundError("Add the original CSVs before running: " + ", ".join(missing))
patients = pd.read_csv(paths[0], dtype={"patient_id": "string"})
outcomes = pd.read_csv(paths[1], dtype={"patient_id": "string"})
patient_required = ["patient_id", "age", "bmi", "systolic_bp", "diastolic_bp", "heart_rate", "charlson_i
ndex"]
outcome_required = ["patient_id", "admission_date", "discharge_date", "primary_drg",
                    "readmitted_30d", "in_hospital_death", "icu_admission",
                    "length_of_stay_days", "total_charges_usd"]
for name, frame, required in [("patients", patients, patient_required), ("outcomes", outcomes, outcome_r
equired)]:
    absent = sorted(set(required) - set(frame.columns))
    if absent:
        raise ValueError(f"{name}: missing columns {absent}")
    if frame["patient_id"].isna().any() or frame["patient_id"].duplicated().any():
        raise ValueError(f"{name}: patient_id must be complete and unique. Repeated admissions require a
different analysis unit.")

dx_cols = sorted(c for c in patients if c.startswith("dx_"))
if not dx_cols:
    raise ValueError("No diagnosis indicators (dx_ columns) found.")
for col in patient_required[1:] + dx_cols:
    patients[col] = pd.to_numeric(patients[col], errors="raise")
for col in ["readmitted_30d", "in_hospital_death", "icu_admission", "length_of_stay_days", "total_charge
s_usd"]:
    outcomes[col] = pd.to_numeric(outcomes[col], errors="raise")
for frame, cols in [(patients, dx_cols), (outcomes, ["readmitted_30d", "in_hospital_death", "icu_admissi
on"])]):
    for col in cols:
        if not frame[col].dropna().isin([0, 1]).all():
            raise ValueError(f"{col} must contain only 0, 1 or missing values.")
for col in ["admission_date", "discharge_date"]:
    outcomes[col] = pd.to_datetime(outcomes[col], errors="raise")
if outcomes[["admission_date", "discharge_date"]].isna().any().any():
    raise ValueError("Admission/discharge dates must be complete.")
if outcomes["discharge_date"] < outcomes["admission_date"].any():
    raise ValueError("Discharge precedes admission.")
if not outcomes["admission_date"].dt.year.between(2018, 2024).all():
    raise ValueError("Admission dates fall outside the intended 2018–2024 window.")
if (outcomes["length_of_stay_days"].dropna() <= 0).any() or (outcomes["total_charges_usd"].dropna() < 0)
.any():
    raise ValueError("Review nonpositive stays or negative charges.")
# Soft check: stay length should agree with the admission/discharge dates (warn, do not stop).
date_lo = (outcomes["discharge_date"] - outcomes["admission_date"]).dt.days
los_gap = (date_lo - outcomes["length_of_stay_days"]).abs()
if (los_gap > 1).any():
    warnings.warn(f"{int((los_gap > 1).sum())} records have length_of_stay_days differing from the date
difference by more than 1 day.")
display(pd.DataFrame({
    "table": ["patients", "outcomes"],
    "rows": [len(patients), len(outcomes)],
    "missing_cells": [patients.isna().sum().sum(), outcomes.isna().sum().sum()]
}))
# Missing days_to_readmission may be structural; it is not required for this analysis.
```

	table	rows	missing_cells
0	patients	100000	0
1	outcomes	6820	5814

1.1. Full-population context: BMI, age, and recorded diagnoses

The following two visualizations describe all records in patients, not only hospitalized adults aged 45–64. They provide context; subsequent analyses use the target hospitalized cohort.

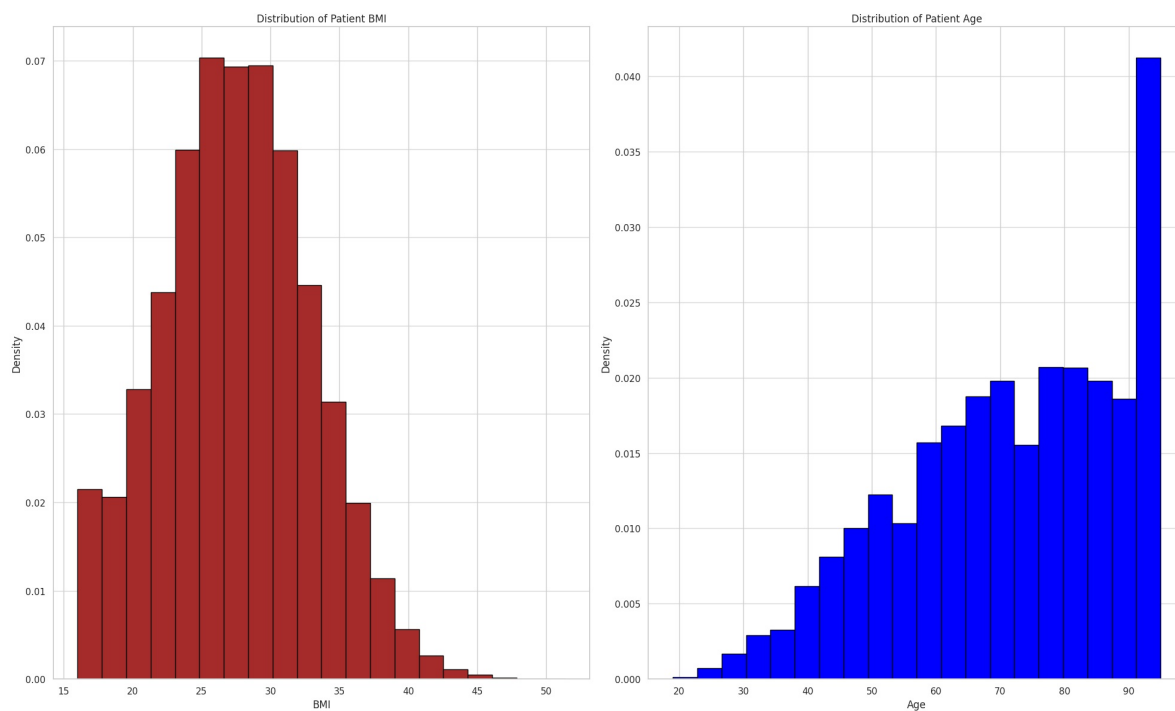
In [4]:

```
import matplotlib.pyplot as plt
#Subplot of BMI and Age
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 12))

# Plot BMI
axes[0].hist(patients['bmi'], density=True, bins=20, edgecolor='black', color='brown')
axes[0].set_title('Distribution of Patient BMI')
axes[0].set_xlabel('BMI')
axes[0].set_ylabel('Density')
axes[0].grid(axis='y', alpha=0.75)

# Plot Age
axes[1].hist(patients['age'], density=True, bins=20, edgecolor='black', color='blue')
axes[1].set_title('Distribution of Patient Age')
axes[1].set_xlabel('Age')
axes[1].set_ylabel('Density')
axes[1].grid(axis='y', alpha=0.75)

plt.tight_layout()
plt.show()
```



In [5]:

```
import matplotlib.pyplot as plt
import seaborn as sns

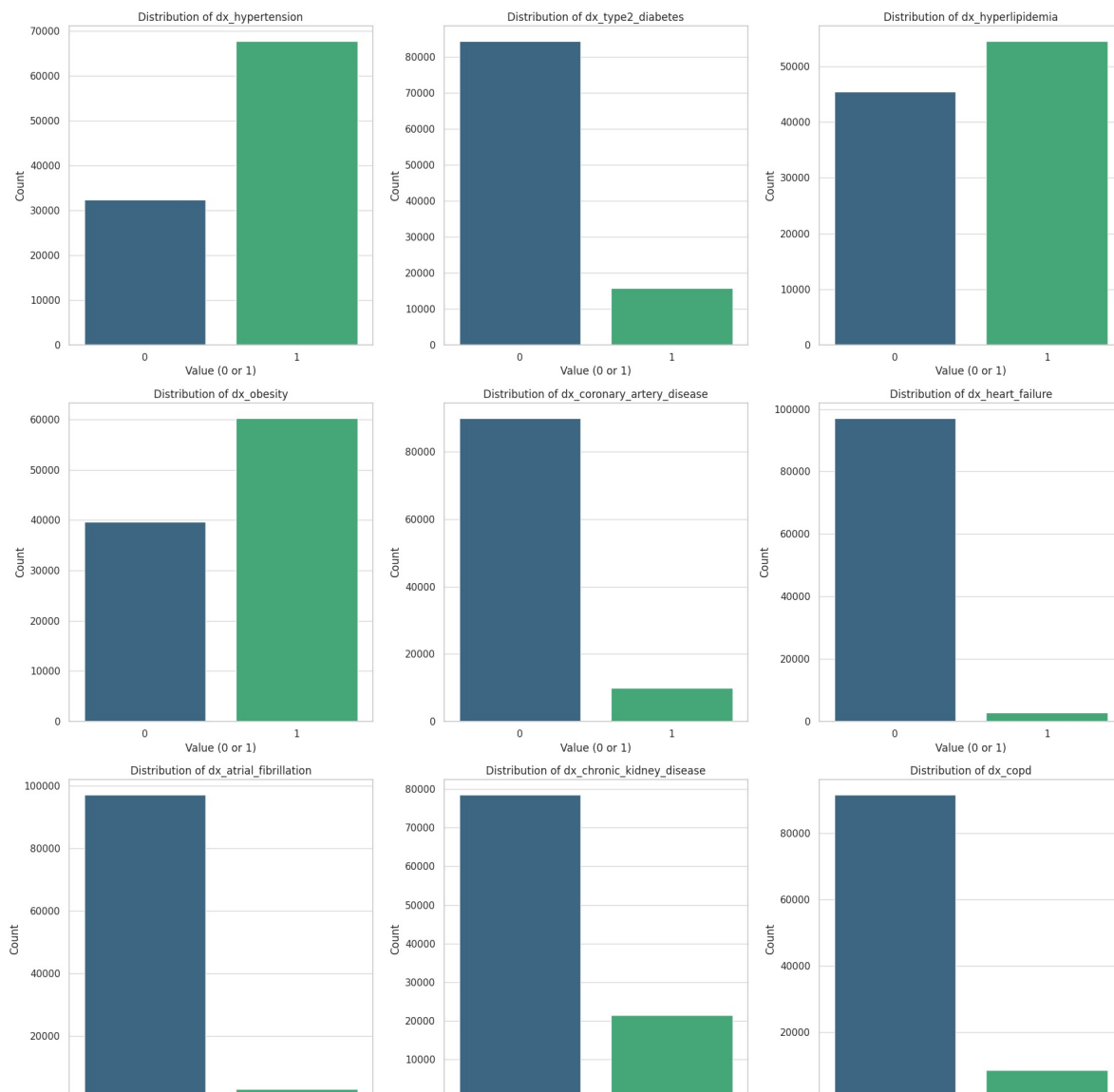
# Identify columns starting with 'dx_'
dx_columns = [col for col in patients.columns if col.startswith('dx_')]

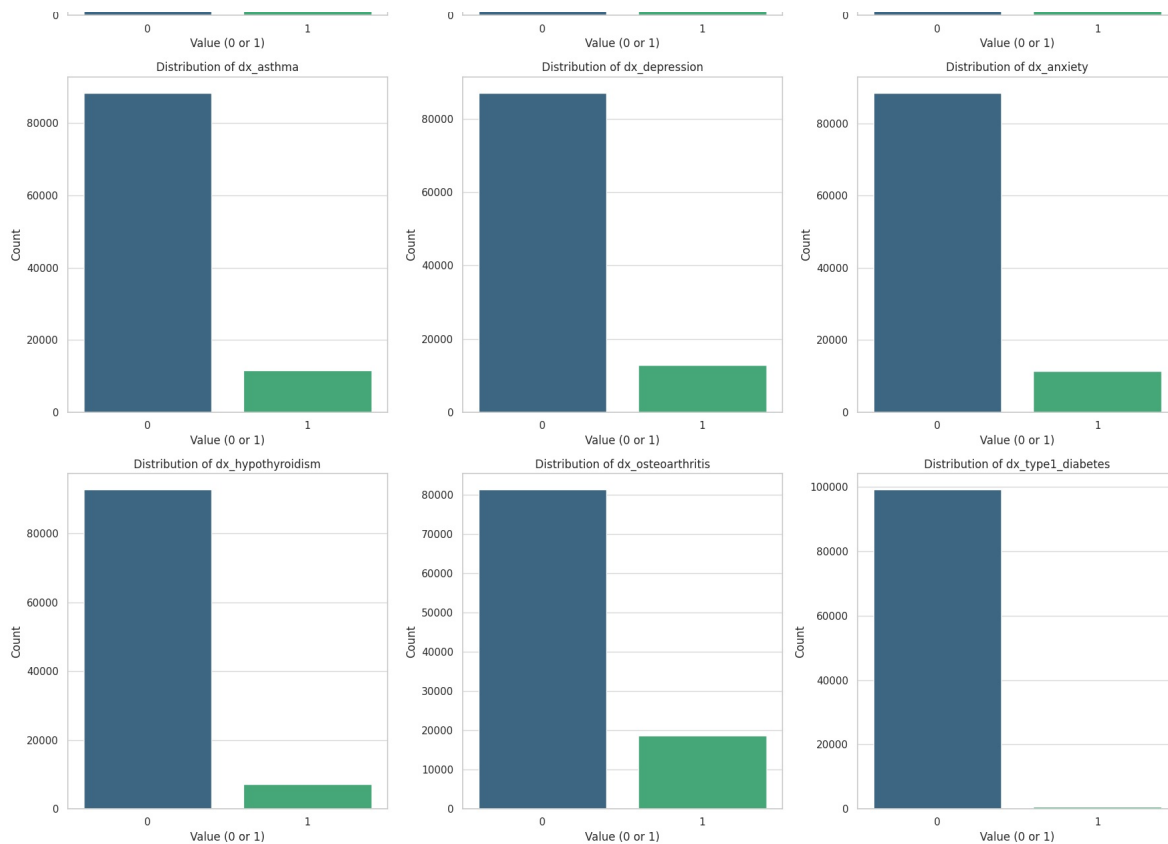
# Determine grid size for subplots
num_cols = len(dx_columns)
num_rows = (num_cols + 2) // 3 # Aim for 3 plots per row, adjust rows accordingly

plt.figure(figsize=(18, 6 * num_rows))

for i, col in enumerate(dx_columns):
    plt.subplot(num_rows, 3, i + 1)
    counts = patients[col].value_counts().sort_index()
    # Updated sns.barplot call to address FutureWarning
    sns.barplot(x=counts.index, y=counts.values, hue=counts.index, palette='viridis', legend=False)
    plt.title(f'Distribution of {col}')
    plt.xlabel('Value (0 or 1)')
    plt.ylabel('Count')
    plt.xticks([0, 1]) # Ensure only 0 and 1 are shown as ticks
    plt.grid(axis='y', alpha=0.75)

plt.tight_layout()
plt.show()
```





2. Cohort definition and focused feature engineering

Retain hospital records whose linked age is 45–64 inclusive. Age is the supplied patient-table age. **Limitation:** it is not confirmed to be age at admission. With admissions spanning 2018–2024, a static age could misclassify some patients into or out of the 45–64 band.

The blood-pressure grouping applies numeric thresholds with the highest applicable band checked first. It is a descriptive measurement grouping, not a clinical diagnosis. Missing diagnoses are not silently counted as absence. DRG codes remain categorical and are used only to describe clusters.

In [6]:

```
patients["condition_count"] = patients[dx_cols].sum(axis=1, min_count=len(dx_cols))
# Exclude a recorded hypertension diagnosis from the comparison outcome to reduce circularity.
other_dx = [c for c in dx_cols if "hypertension" not in c.lower()]
patients["other_condition_count"] = patients[other_dx].sum(axis=1, min_count=len(other_dx))
s, d = patients["systolic_bp"], patients["diastolic_bp"]
patients["bp_group"] = np.select(
    [s.isna() | d.isna(), (s > 180) | (d > 120), (s >= 140) | (d >= 90),
     (s >= 130) | (d >= 80), (s >= 120)],
    ["Missing", "Highest band", "Stage 2 range", "Stage 1 range", "Elevated range"],
    default="Normal range"
)
joined = outcomes.merge(patients, on="patient_id", how="left", validate="one_to_one", indicator=True)
if not joined["_merge"].eq("both").all():
    raise ValueError("Some hospital records have no matching patient record.")
joined = joined.drop(columns="_merge")
if joined["age"].isna().any():
    raise ValueError("Age must be complete to define the cohort.")
adult = joined.loc[joined["age"].between(AGE_MIN, AGE_MAX)].copy().reset_index(drop=True)
senior = joined.loc[joined["age"] >= 65].copy()
if len(adult) < 20:
    raise ValueError("Too few cohort records for the proposed segmentation.")
adult["primary_drg"] = adult["primary_drg"].astype("string")
display(pd.Series({
    "all_patient_records": len(patients),
    "all_hospital_records": len(joined),
    "hospitalized_45_64": len(adult),
    "share_of_hospital_records_pct": 100 * len(adult) / len(joined)
}, name="Cohort counts").to_frame())
```

Cohort counts	
all_patient_records	100000.000000
all_hospital_records	6820.000000
hospitalized_45_64	985.000000
share_of_hospital_records_pct	14.442815

2.1. Additional descriptive features

Retain admission year, BMI bands, and charges per hospital day for cohort description. BMI bands use left-inclusive thresholds, so exactly 25 belongs to the 25-<30 band. These bands are descriptive and not diagnoses. Condition count, BP grouping, and age filtering already exist above. No lifestyle ordinal codes, DRG numerical encoding, or seasonal features are needed for the current clustering question.

In [7]:

```
# Descriptive features only: clustering inputs remain unchanged.
adult['admission_year'] = adult['admission_date'].dt.year
adult['bmi_band'] = pd.cut(
    adult['bmi'], bins=[-np.inf, 18.5, 25, 30, 40, np.inf],
    labels=['<18.5', '18.5-<25', '25-<30', '30-<40', '40+'], right=False
)
# This is billed charges per day, not provider cost per day.
adult['charges_per_day'] = adult['total_charges_usd'].div(
    adult['length_of_stay_days'].where(adult['length_of_stay_days'] > 0)
)
display(adult['bmi_band'].value_counts(sort=False, dropna=False).rename('patients').to_frame())
display(adult[['condition_count', 'other_condition_count', 'charges_per_day']].describe().T)
```

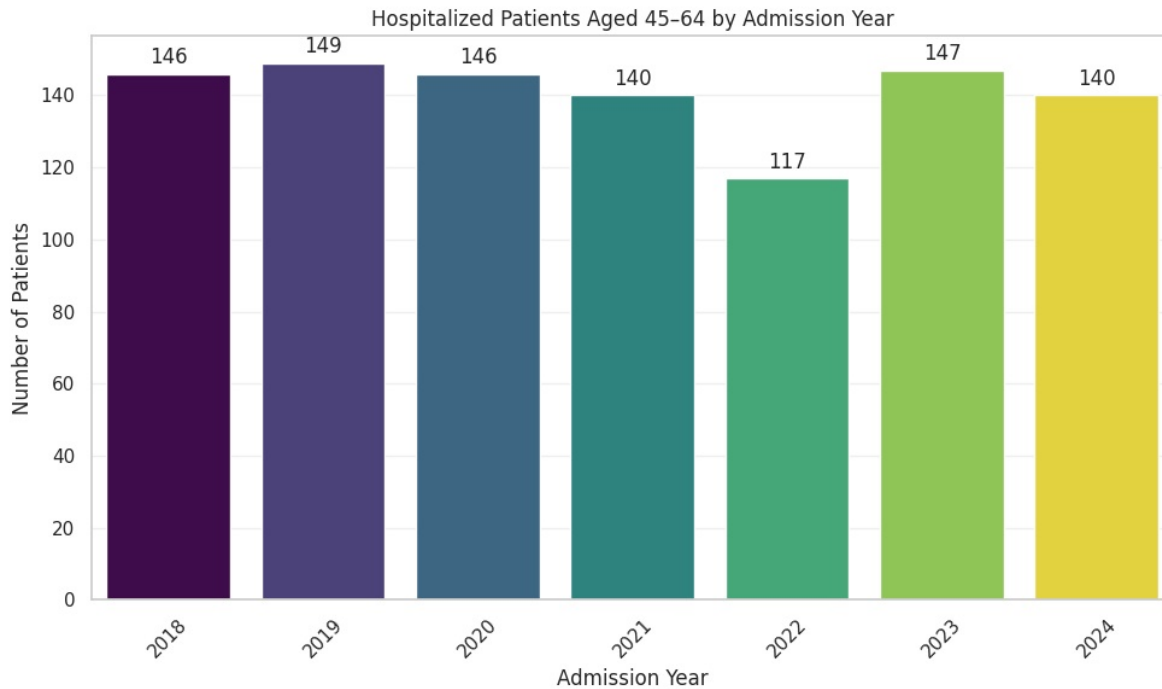
patients									
bmi_band									
<18.5	53								
18.5-<25	266								
25-<30	338								
30-<40	322								
40+	6								
		count	mean	std	min	25%	50%	75%	max
condition_count		985.0	2.819289	1.343395	0.000000	2.00	3.000	4.000	7.00
other_condition_count		985.0	2.276142	1.248938	0.000000	1.00	2.000	3.000	6.00
charges_per_day		985.0	5924.250912	7467.104302	100.551379	1661.55	3331.605	6727.974	64721.99

2.2. Admissions by year: hospitalized adults aged 45-64

This chart uses `adult` rather than `all_outcomes`. Because the validated data contain one hospital record per patient, counts represent both records and unique patients. They are sample counts, not population hospitalization rates; zero means no observed records for that year.

In [8]:

```
yearly_admissions = (
    adult['admission_year'].value_counts()
    .reindex(range(2018, 2025), fill_value=0).sort_index()
)
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=yearly_admissions.index, y=yearly_admissions.values,
            hue=yearly_admissions.index, palette='viridis', legend=False, ax=ax)
ax.set(title='Hospitalized Patients Aged 45-64 by Admission Year',
        xlabel='Admission Year', ylabel='Number of Patients')
ax.tick_params(axis='x', rotation=45)
ax.grid(axis='y', alpha=0.3)
for container in ax.containers:
    ax.bar_label(container, fmt='%0f', padding=3)
plt.tight_layout()
plt.show()
display(yearly_admissions.rename('patients').to_frame())
```



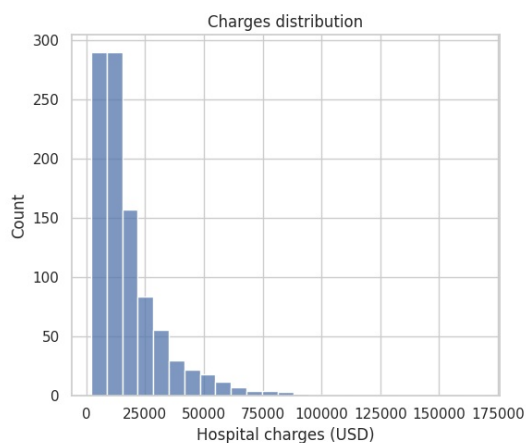
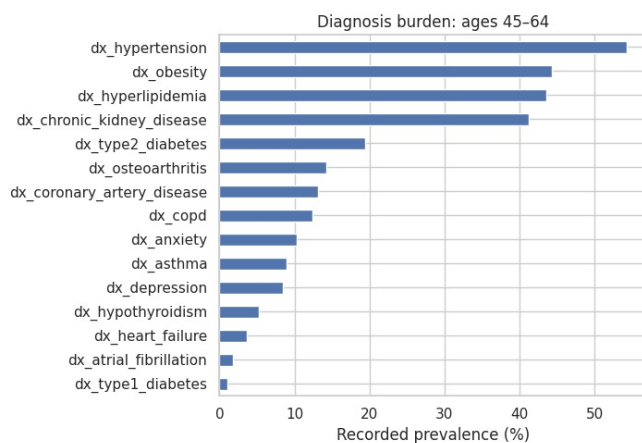
patients	
admission_year	
2018	146
2019	149
2020	146
2021	140
2022	117
2023	147
2024	140

In [9]:

```
outcome_cols = ["readmitted_30d", "in_hospital_death", "icu_admission",
                "length_of_stay_days", "total_charges_usd"]
display(adult[["age", "bmi", "charlson_index", "condition_count"] + outcome_cols].describe().T)
display(adult[outcome_cols].isna().sum().rename("missing_outcomes").to_frame())
prevalence = adult[dx_cols].mean().sort_values(ascending=False).mul(100)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
prevalence.sort_values().plot.barh(ax=axes[0], color="#5275ad")
axes[0].set(xlabel="Recorded prevalence (%)", ylabel="", title="Diagnosis burden: ages 45-64")
sns.histplot(adult["total_charges_usd"].dropna(), bins=25, ax=axes[1], color="#5275ad")
axes[1].set(xlabel="Hospital charges (USD)", title="Charges distribution")
plt.tight_layout()
plt.show()
# Denominators vary when an outcome is missing; retain outcome-specific counts.
display(adult[outcome_cols].agg(["count", "mean", "median"]).T)
```

	count	mean	std	min	25%	50%	75%	max
age	985.0	55.857868	5.513666	45.0	52.00	56.00	61.00	64.00
bmi	985.0	27.441726	5.436712	16.0	23.70	27.20	31.50	41.00
charlson_index	985.0	1.329949	1.082833	0.0	0.00	1.00	2.00	5.00
condition_count	985.0	2.819289	1.343395	0.0	2.00	3.00	4.00	7.00
readmitted_30d	985.0	0.116751	0.321287	0.0	0.00	0.00	0.00	1.00
in_hospital_death	985.0	0.011168	0.105138	0.0	0.00	0.00	0.00	1.00
icu_admission	985.0	0.016244	0.126475	0.0	0.00	0.00	0.00	1.00
length_of_stay_days	985.0	5.157360	4.095234	1.0	2.00	4.00	7.00	30.00
total_charges_usd	985.0	17994.686589	16793.599837	2000.0	7665.11	12772.89	21888.71	167283.86

missing_outcomes	
readmitted_30d	0
in_hospital_death	0
icu_admission	0
length_of_stay_days	0
total_charges_usd	0



	count	mean	median
readmitted_30d	985.0	0.116751	0.00
in_hospital_death	985.0	0.011168	0.00
icu_admission	985.0	0.016244	0.00
length_of_stay_days	985.0	5.157360	4.00
total_charges_usd	985.0	17994.686589	12772.89

3. Focused observational comparisons

These are **not randomized A/B tests**. Both comparisons are exploratory and two-sided.

- Within the target cohort: other diagnosis count for Stage 1-range versus normal-range measured BP.
- Context only: hospital charges for adults 45-64 versus adults 65+.

Report differences, uncertainty and sample sizes, rather than treating nonsignificance as proof of equivalence. Neither comparison adjusts for case mix or establishes causality.

In [10]:

```
def bootstrap_mean_difference(a, b, seed=SEED, n_boot=N_BOOT):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    a, b = a[np.isfinite(a)], b[np.isfinite(b)]
    if min(len(a), len(b)) < 2:
        raise ValueError("Each comparison group needs at least two valid observations.")
    rng = np.random.default_rng(seed)
    diffs = np.empty(n_boot)
    for i in range(n_boot):
        diffs[i] = rng.choice(a, len(a), replace=True).mean() - rng.choice(b, len(b), replace=True).mean()
    return a.mean() - b.mean(), *np.quantile(diffs, [0.025, 0.975])

comparisons = [
    ("Stage 1 minus normal BP: other diagnoses",
     adult.loc[adult["bp_group"].eq("Stage 1 range"), "other_condition_count"],
     adult.loc[adult["bp_group"].eq("Normal range"), "other_condition_count"]),
    ("Age 45-64 minus age 65+: charges USD",
     adult["total_charges_usd"], senior["total_charges_usd"])
]
test_rows = []
for label, a, b in comparisons:
    a, b = a.dropna(), b.dropna()
    if min(len(a), len(b)) < 2:
        print(f"Skipped {label}: insufficient observations.")
        continue
    difference, low, high = bootstrap_mean_difference(a, b)
    result = stats.ttest_ind(a, b, equal_var=False, alternative="two-sided")
    test_rows.append(dict(comparison=label, n_a=len(a), n_b=len(b),
                          difference=difference, ci_low=low, ci_high=high, p_value=result.pvalue))
tests = pd.DataFrame(test_rows)
if not tests.empty:
    tests["p_bonferroni"] = (tests["p_value"] * len(tests)).clip(upper=1)
display(tests)
# Intervals are unadjusted exploratory percentile bootstrap intervals.
```

	comparison	n_a	n_b	difference	ci_low	ci_high	p_value	p_bonferroni
0	Stage 1 minus normal BP: other diagnoses	322	151	0.174407	-0.067793	0.408345	0.157522	0.315045
1	Age 45-64 minus age 65+: charges USD	985	5579	-495.937044	-1509.474522	665.514266	0.395113	0.790226

Reading the comparisons. Stage 1-range patients had about 0.17 more other diagnoses than normal-range patients (95% bootstrap interval -0.07 to 0.41 ; $n=322$ vs 151). Mean charges for ages 45-64 were about \$496 lower than for ages 65+ (interval $-\$1,509$ to $+\$666$; $n=985$ vs $5,579$). Both intervals include zero, and the Bonferroni-adjusted p-values (0.32 and 0.79) give no evidence of a difference. This does not show the groups are equivalent, and neither comparison adjusts for case mix. Charges are right-skewed, so differences in means should be read cautiously.

4. Outcome uncertainty and a clearly labeled planning scenario

Bootstrap at the **observed cohort size** to estimate uncertainty around observed rates and mean charges. Separately, simulate an assumed 5,500-patient cohort at the observed event probability. The latter is a conditional scenario interval, **not** a confidence interval for the observed rate and **not** a city-level or annual forecast.

Readmission uses all nonmissing recorded indicators. The source definition must establish whether deaths and other ineligible discharges are included. Rare mortality events require cautious interpretation.

In [11]:

```
def bootstrap_mean(values, seed=SEED, n_boot=N_BOOT):
    x = np.asarray(pd.Series(values).dropna(), dtype=float)
    if len(x) < 2:
        return np.nan, np.nan, np.nan
    rng = np.random.default_rng(seed)
    means = np.array([rng.choice(x, size=len(x), replace=True).mean() for _ in range(n_boot)])
    return x.mean(), *np.quantile(means, [0.025, 0.975])

uncertainty_rows = []
for i, col in enumerate(outcome_cols):
    estimate, low, high = bootstrap_mean(adult[col], seed=SEED+i)
    uncertainty_rows.append(dict(metric=col, n=adult[col].count(), estimate=estimate, ci_low=low, ci_high=high))
uncertainty = pd.DataFrame(uncertainty_rows)
display(uncertainty)

# Illustrative planning assumption only; replace with the real expected admission volume before using.
ASSUMED_COHORT_SIZE = 5500
rng = np.random.default_rng(SEED)
scenario_rows = []
for col in ["readmitted_30d", "in_hospital_death"]:
    p = adult[col].mean()
    if pd.isna(p):
        continue
    simulated_counts = rng.binomial(ASSUMED_COHORT_SIZE, p, size=10000)
    lo, hi = np.quantile(simulated_counts, [0.025, 0.975])
    scenario_rows.append(dict(outcome=col, assumed_patients=ASSUMED_COHORT_SIZE,
                              observed_probability=p, expected_count=ASSUMED_COHORT_SIZE*p,
                              scenario_count_low=lo, scenario_count_high=hi))
display(pd.DataFrame(scenario_rows))
# This fixed-probability scenario excludes parameter uncertainty, temporal drift, and case-mix changes.
```

	metric	n	estimate	ci_low	ci_high
0	readmitted_30d	985	0.116751	0.097462	0.137056
1	in_hospital_death	985	0.011168	0.005076	0.018274
2	icu_admission	985	0.016244	0.008122	0.024365
3	length_of_stay_days	985	5.157360	4.899416	5.409137
4	total_charges_usd	985	17994.686589	16999.963810	19034.911561

	outcome	assumed_patients	observed_probability	expected_count	scenario_count_low	scenario_count_high
0	readmitted_30d	5500	0.116751	642.13198	596.0	
1	in_hospital_death	5500	0.011168	61.42132	46.0	

5. PCA on the target hospitalized cohort

Fit separate PCA blocks for continuous clinical measurements and binary diagnosis indicators. Retain 80% and 75% of variance respectively rather than silently resetting both blocks to four components.

Exclude hospital outcomes, DRG codes, and aggregate diagnosis counts from clustering. This avoids outcome leakage, artificial ordering of DRGs, and duplicating diagnosis information through summary scores. Lifestyle variables remain available for profiling but are not arbitrarily ordinal-encoded.

Use complete baseline records and explicitly report exclusions. Standardized PCA on binary diagnoses is exploratory: rare diagnoses can receive high weight, and alternative mixed-data methods are a future sensitivity analysis.

In [12]:

```
continuous_features = ["age", "bmi", "systolic_bp", "diastolic_bp", "heart_rate"]
baseline_features = continuous_features + dx_cols
complete = adult[baseline_features].replace([np.inf, -np.inf], np.nan).notna().all(axis=1)
cluster_base = adult.loc[complete].copy()
print(f"Clustering: {len(cluster_base):,} retained; {(~complete).sum():,} excluded for incomplete baseline features.")
if len(cluster_base) < 20:
    raise ValueError("Too few complete baseline records.")

def fit_pca_block(frame, features, variance_target, prefix):
    varying = [c for c in features if frame[c].nunique() > 1]
    if not varying:
        raise ValueError(f"{prefix}: no varying features.")
    scaler = StandardScaler()
    scaled = scaler.fit_transform(frame[varying])
    pca = PCA(n_components=variance_target, svd_solver="full")
    scores = pca.fit_transform(scaled)
    names = [f"{prefix}_PC{i+1}" for i in range(scores.shape[1])]
    # Unit total variance per block: equal weight for clinical and diagnosis blocks.
    balanced_scores = scores / np.sqrt(pca.explained_variance_.sum())
    loadings = pd.DataFrame(pca.components_.T, index=varying, columns=names)
    print(f"{prefix}: {pca.n_components_} components, {pca.explained_variance_ratio_.sum():.1%} variance retained")
    display(loadings.round(3))
    return balanced_scores, pca, scaler, loadings

clinical_scores, clinical_pca, clinical_scaler, clinical_loadings = fit_pca_block(
    cluster_base, continuous_features, 0.80, "clinical")
disease_scores, disease_pca, disease_scaler, disease_loadings = fit_pca_block(
    cluster_base, dx_cols, 0.75, "diagnosis")
X = np.column_stack([clinical_scores, disease_scores])
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, model, title in zip(axes, [clinical_pca, disease_pca], ["Clinical measurements", "Diagnoses"]):
    ax.plot(range(1, model.n_components_+1), np.cumsum(model.explained_variance_ratio_), marker="o")
    ax.set(xlabel="Retained components", ylabel="Cumulative variance explained", title=title, ylim=(0, 1.05))
plt.tight_layout()
plt.show()
```

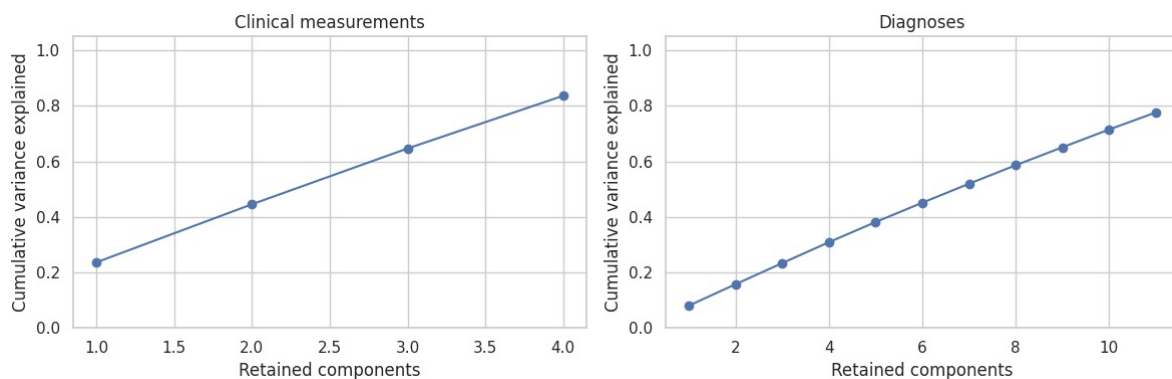
Clustering: 985 retained; 0 excluded for incomplete baseline features.

clinical: 4 components, 83.6% variance retained

	clinical_PC1	clinical_PC2	clinical_PC3	clinical_PC4
age	0.695	0.078	-0.081	0.159
bmi	-0.100	0.726	0.014	0.667
systolic_bp	0.622	0.252	-0.279	-0.299
diastolic_bp	0.341	-0.478	0.547	0.492
heart_rate	0.057	0.418	0.785	-0.445

diagnosis: 11 components, 77.6% variance retained

	diagnosis_PC1	diagnosis_PC2	diagnosis_PC3	diagnosis_PC4	diagnosis_PC5	diag
dx_anxiety	-0.202	-0.325	0.071	0.139	0.173	
dx_asthma	-0.294	-0.055	0.455	-0.170	-0.005	
dx_atrial_fibrillation	-0.077	-0.132	0.359	-0.193	0.266	
dx_chronic_kidney_disease	0.584	-0.297	-0.081	-0.008	0.115	
dx_copd	-0.133	0.622	0.132	-0.014	0.137	
dx_coronary_artery_disease	-0.192	0.059	0.519	0.139	-0.039	
dx_depression	-0.016	-0.027	0.093	0.561	0.154	
dx_heart_failure	-0.209	0.304	-0.348	0.053	-0.215	
dx_hyperlipidemia	-0.298	0.005	-0.295	0.082	0.278	
dx_hypertension	-0.067	0.075	-0.171	-0.342	0.601	
dx_hypothyroidism	-0.245	-0.256	-0.105	0.078	-0.430	
dx_obesity	-0.199	0.020	-0.216	0.477	0.292	
dx_osteoarthritis	0.198	0.079	0.209	0.451	0.007	
dx_type1_diabetes	-0.179	0.090	-0.085	-0.124	-0.289	
dx_type2_diabetes	-0.403	-0.465	-0.109	-0.041	0.054	



6. K-means selection and stability

Compare $k=2-10$ using silhouette, minimum cluster size and repeat-initialization stability (adjusted Rand index). Select the highest-silhouette solution among those with at least 20 patients and 5% of the cohort per cluster. Seed-stability ARI is reported alongside but is **not** part of the selection rule; treat solutions with ARI below ~ 0.8 as unstable. The size thresholds are explicit analyst choices, not clinical thresholds.

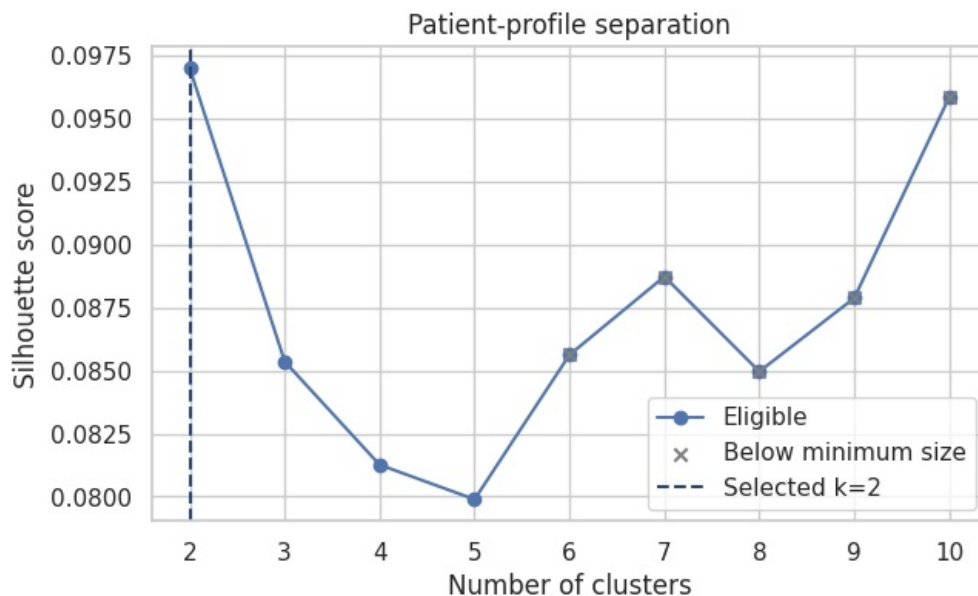
$k=3$ is a **sensitivity comparison only**, not an automatic choice. Weak silhouettes mean overlapping profiles, even when one k ranks highest. Repeated-seed stability measures optimization consistency, not external validity.

In [13]:

```
minimum_size = max(20, int(np.ceil(0.05 * len(X))))
models, rows = {}, []
for k in range(2, min(10, len(X)-1) + 1):
    model = KMeans(n_clusters=k, n_init=30, random_state=SEED).fit(X)
    labels = model.labels_
    if len(np.unique(labels)) < k:
        continue
    sil = silhouette_score(X, labels)
    stability = []
    for offset in [1, 2, 3]:
        alternate = KMeans(n_clusters=k, n_init=10, random_state=SEED+offset).fit_predict(X)
        stability.append(adjusted_rand_score(labels, alternate))
    counts = np.bincount(labels)
    rows.append(dict(k=k, silhouette=sil, min_cluster_n=counts.min(),
                    mean_seed_ARI=np.mean(stability), inertia=model.inertia_,
                    eligible=counts.min() >= minimum_size))
    models[k] = model
selection = pd.DataFrame(rows).sort_values("k")
if selection.empty:
    raise ValueError("No valid clustering solution; inspect baseline feature variation.")
display(selection)
eligible = selection.loc[selection["eligible"]]
if eligible.empty:
    raise ValueError("No solution meets the declared minimum-size rule; review features before changing it.")
best_k = int(eligible.sort_values(["silhouette", "k"], ascending=[False, True]).iloc[0]["k"])
best_model = models[best_k]
print(f"Selected k={best_k}; minimum segment size rule={minimum_size}")
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(selection["k"], selection["silhouette"], marker="o", label="Eligible")
inel = selection.loc[~selection["eligible"]]
ax.scatter(inel["k"], inel["silhouette"], marker="x", color="grey", zorder=3, label="Below minimum size")
ax.axvline(best_k, color="#284570", linestyle="--", label=f"Selected k={best_k}")
ax.set(xlabel="Number of clusters", ylabel="Silhouette score", title="Patient-profile separation")
ax.legend()
plt.show()
if 3 in models and best_k != 3:
    print("k=3 sensitivity: compare separation, size and seed stability in the table before preferring i
nterpretability over fit.")
    display(pd.crosstab(best_model.labels_, models[3].labels_, rownames=["Selected solution"], colnames=
["k=3 solution"])))
```

	k	silhouette	min_cluster_n	mean_seed_ARI	inertia	eligible
0	2	0.097006	459	0.967845	1775.617918	True
1	3	0.085362	314	0.817679	1666.150248	True
2	4	0.081262	214	0.421829	1586.424657	True
3	5	0.079890	181	0.315875	1521.032763	True
4	6	0.085626	10	0.478893	1445.150320	False
5	7	0.088735	10	0.492772	1389.718252	False
6	8	0.084965	10	0.327744	1348.887947	False
7	9	0.087883	10	0.272328	1296.580597	False
8	10	0.095853	10	0.249164	1270.895765	False

Selected k=2; minimum segment size rule=50



k=3 sensitivity: compare separation, size and seed stability in the table before preferring interpretability over fit.

k=3 solution	0	1	2
Selected solution			
0	44	63	352
1	272	251	3

7. Patient profiles and held-out outcome descriptions

Outcomes are held out of feature construction, **not** held out as an independent validation sample. This is descriptive profiling of the same cohort. Cluster IDs have no intrinsic meaning and may change after rerunning or changing preprocessing.

Report event counts and denominators alongside rates. Wilson intervals avoid implying certainty when a cluster has zero observed events. Do not infer disease stage, progression, treatment effectiveness or medication recommendations from a diagnosis flag.

In [14]:

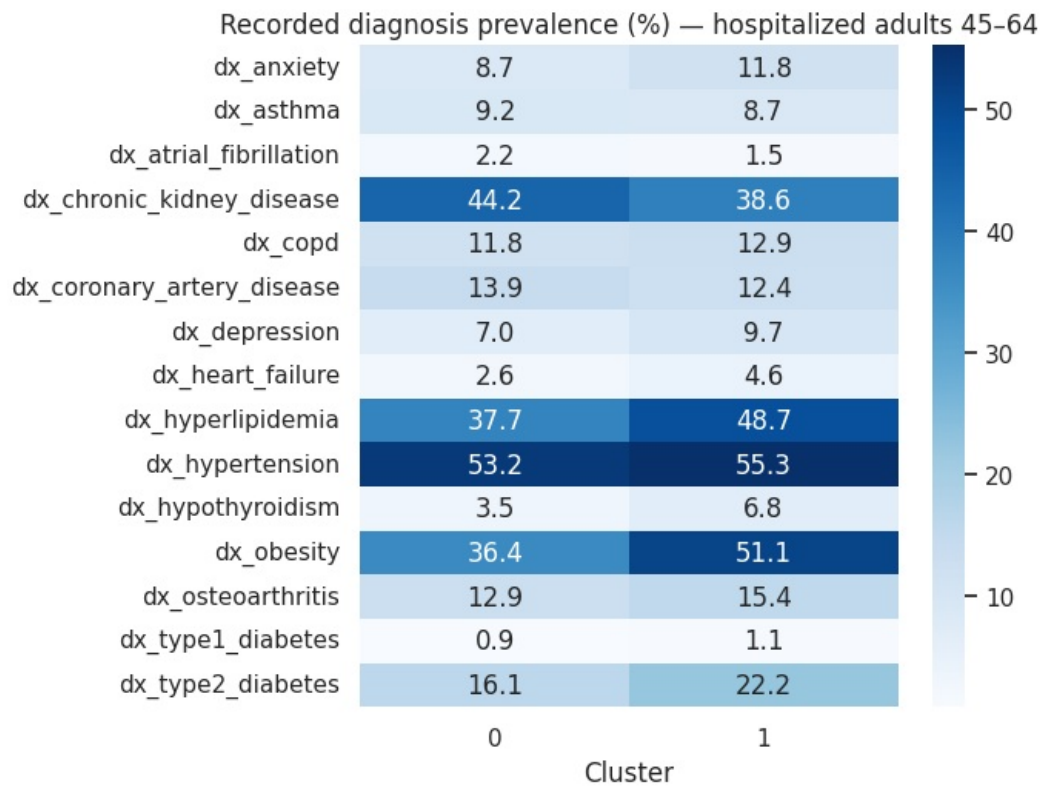
```
clustered = cluster_base.copy()
clustered["cluster"] = best_model.labels_
profile = clustered.groupby("cluster")[continuous_features + ["charlson_index", "condition_count"]].mean()
display(profile.round(2))
dx_profile = clustered.groupby("cluster")[dx_cols].mean().mul(100)
fig, ax = plt.subplots(figsize=(max(7, best_k*1.4), max(5, len(dx_cols)*0.36)))
sns.heatmap(dx_profile.T, annot=True, fmt=".1f", cmap="Blues", ax=ax)
ax.set(title="Recorded diagnosis prevalence (%) - hospitalized adults 45-64", xlabel="Cluster", ylabel="")
plt.tight_layout()
plt.show()
display(pd.crosstab(clustered["cluster"], clustered["primary_drg"], normalize="index").mul(100).round(1))
for col in ["smoking_status", "exercise_level", "insurance_type"]:
    if col in clustered:
        print(col)
        display(pd.crosstab(clustered["cluster"], clustered[col], normalize="index").round(3))

def wilson_interval(events, n, confidence=0.95):
    if n == 0:
        return np.nan, np.nan
    z = stats.norm.ppf(0.5 + confidence/2)
    p, denom = events/n, 1 + z*z/n
    center = (p + z*z/(2*n)) / denom
    half = z * np.sqrt(p*(1-p)/n + z*z/(4*n*n)) / denom
    return center-half, center+half

event_rows, resource_rows = [], []
for cluster, group in clustered.groupby("cluster"):
    for metric in ["readmitted_30d", "in_hospital_death", "icu_admission"]:
        observed = group[metric].dropna()
        events, n = int(observed.sum()), len(observed)
        low, high = wilson_interval(events, n)
        event_rows.append(dict(cluster=cluster, metric=metric, events=events, n=n,
                               rate=events/n if n else np.nan, ci_low=low, ci_high=high))
    resource_rows.append(dict(cluster=cluster, n_patients=len(group),
                              mean_charges=group["total_charges_usd"].mean(),
                              median_charges=group["total_charges_usd"].median(),
                              mean_stay_days=group["length_of_stay_days"].mean(),
                              n_charges=group["total_charges_usd"].count(),
                              n_stay=group["length_of_stay_days"].count()))
event_profile = pd.DataFrame(event_rows)
resource_profile = pd.DataFrame(resource_rows)
display(event_profile.round(4))
display(resource_profile.round(2))

readmit = event_profile.loc[event_profile["metric"].eq("readmitted_30d")].dropna(subset=["rate"])
fig, ax = plt.subplots(figsize=(7, 4))
ax.errorbar(readmit["cluster"], readmit["rate"]*100,
            yerr=np.maximum(0, np.vstack([readmit["rate"]-readmit["ci_low"],
                                           readmit["ci_high"]-readmit["rate"]])*100),
            fmt="o", capsize=5, color="#5275ad")
ax.set(xlabel="Cluster", ylabel="Recorded readmission rate (%)", title="Readmission with 95% Wilson intervals")
ax.set_xticks(readmit["cluster"])
plt.show()
```

	age	bmi	systolic_bp	diastolic_bp	heart_rate	charlson_index	condition_count
cluster							
0	51.95	27.57	117.47	77.84	70.75	1.35	2.60
1	59.27	27.33	136.54	83.47	72.58	1.32	3.01



primary_drg 194 291 292 293 392 470

cluster						
0	14.4	21.1	17.9	16.1	14.2	16.3
1	13.3	20.3	21.3	12.9	18.1	14.1

smoking_status

smoking_status current former never

cluster			
0	0.181	0.275	0.545
1	0.150	0.245	0.605

exercise_level

exercise_level light moderate sedentary vigorous

cluster				
0	0.386	0.266	0.214	0.135
1	0.397	0.272	0.255	0.076

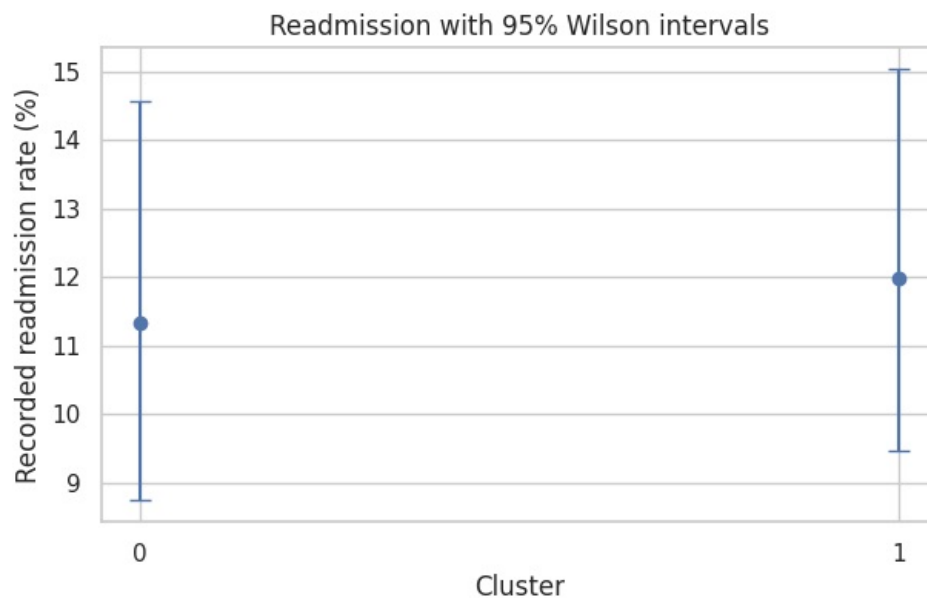
insurance_type

insurance_type commercial medicaid medicare tricare uninsured

cluster					
0	0.471	0.207	0.185	0.052	0.085
1	0.496	0.154	0.179	0.051	0.120

	cluster	metric	events	n	rate	ci_low	ci_high
0	0	readmitted_30d	52	459	0.1133	0.0874	0.1456
1	0	in_hospital_death	4	459	0.0087	0.0034	0.0222
2	0	icu_admission	5	459	0.0109	0.0047	0.0252
3	1	readmitted_30d	63	526	0.1198	0.0947	0.1503
4	1	in_hospital_death	7	526	0.0133	0.0065	0.0272
5	1	icu_admission	11	526	0.0209	0.0117	0.0371

	cluster	n_patients	mean_charges	median_charges	mean_stay_days	n_charges	n_stay
0	0	459	16536.26	12025.38	5.07	459	459
1	1	526	19267.34	13628.77	5.24	526	526



8. Evidence summary

The next cell generates descriptive statements only after the source data have been processed. Treat candidate segments as hypotheses for investigation, not clinically validated risk groups. Cluster IDs and percentages apply only to this run.

In [15]:

```
print(f"Analyzed {len(adult):,} hospitalized adults aged 45–64; {len(clustered):,} had complete clustering features.")
print(f"Selected {best_k} exploratory segments with silhouette {selection.loc[selection.k.eq(best_k), 'silhouette'].iloc[0]:.3f}.")
if not readmit.empty:
    highest = readmit.loc[readmit["rate"].idxmax()]
    print(f"Observed readmission was highest in cluster {int(highest['cluster'])}: "
          f"{highest['rate']:.1%} ({int(highest['events'])}/{int(highest['n'])}); "
          f"95% Wilson interval {highest['ci_low']:.1%}–{highest['ci_high']:.1%}), "
          f"but intervals across clusters overlap substantially.")
for cluster in dx_profile.index:
    top = dx_profile.loc[cluster].nlargest(3)
    print(f"Cluster {cluster}, most prevalent recorded diagnoses: " +
          "; ".join(f"{name.removeprefix('dx_')}: {value:.1f}%" for name, value in top.items()))
print("These are synthetic-data associations, not evidence that an intervention reduces hospitalizations or improves survival.")

selection.to_csv(OUTPUT_DIR / "cluster_selection.csv", index=False)
profile.to_csv(OUTPUT_DIR / "patient_profiles.csv")
dx_profile.to_csv(OUTPUT_DIR / "diagnosis_prevalence_pct.csv")
event_profile.to_csv(OUTPUT_DIR / "cluster_events_and_intervals.csv", index=False)
resource_profile.to_csv(OUTPUT_DIR / "cluster_resource_use.csv", index=False)
uncertainty.to_csv(OUTPUT_DIR / "cohort_uncertainty.csv", index=False)
# Aggregate outputs only; no patient-level records are exported.
```

Analyzed 985 hospitalized adults aged 45–64; 985 had complete clustering features.
Selected 2 exploratory segments with silhouette 0.097.
Observed readmission was highest in cluster 1: 12.0% (63/526; 95% Wilson interval 9.5%–15.0%), but intervals across clusters overlap substantially.
Cluster 0, most prevalent recorded diagnoses: hypertension: 53.2%; chronic_kidney_disease: 44.2%; hyperlipidemia: 37.7%
Cluster 1, most prevalent recorded diagnoses: hypertension: 55.3%; obesity: 51.1%; hyperlipidemia: 48.7%
These are synthetic-data associations, not evidence that an intervention reduces hospitalizations or improves survival.

8.1. Interpretation of the saved focused-model results

Cohort. The hospitalized cohort contains **985 adults aged 45–64**, about **14.4% of the 6,820 hospital records**. All 985 had complete baseline features, so none were excluded from clustering.

Main finding: the data show little evidence of distinct patient segments. K-means selected **k=2** with a silhouette of **0.097**, far below the ~0.25 level usually read as even weak structure. The ranking among solutions is also fragile:

- Among eligible solutions, k=2 beats k=3 (0.085) by only 0.012.
- k=10 scores 0.096, nearly tied with k=2, and was excluded only by the minimum-size rule (smallest cluster of 10 patients).
- Stability worsens quickly with k. Mean seed ARI is 0.97 for k=2 and 0.82 for k=3, but only 0.42 for k=4 and 0.32 for k=5. ARI reflects optimizer consistency, not external validity.

The two clusters should therefore be read as one convenient partition of a largely continuous population, not as discovered patient types.

What separates the clusters. The split is driven mainly by age and blood pressure, not by diagnosis pattern:

	Cluster 0 (n=459)	Cluster 1 (n=526)
Mean age	52.0	59.3
Mean systolic / diastolic BP	117 / 78	137 / 83
Mean Charlson index	1.35	1.32
Mean condition count	2.60	3.01

Age loads most heavily on the first clinical component (0.70, with systolic BP at 0.62). Age also defines the cohort, so part of this segmentation re-splits the cohort along its own selection variable. Charlson index, which was not a clustering input, is essentially identical across clusters.

Diagnosis patterns. Hypertension is common in both clusters (53.2% and 55.3%). Cluster 0 also shows chronic kidney disease (44.2%) and hyperlipidemia (37.7%). Cluster 1 shows obesity (51.1%) and hyperlipidemia (48.7%). Hypertension prevalence is nearly the same even though mean systolic BP differs by about 19 mmHg. This suggests the synthetic diagnosis flags were generated largely independently of the vital signs, so the clusters may carry less clinical meaning than they appear to.

Held-out outcomes. Differences are small and within sampling noise:

Outcome	Cluster 0	Cluster 1
30-day readmission	11.3% (52/459; 95% CI 8.7–14.6%)	12.0% (63/526; 95% CI 9.5–15.0%)
In-hospital death	4 events	7 events
ICU admission	5 events	11 events
Median charges	\$12,025	\$13,629
Mean length of stay	5.07 days	5.24 days

The 0.65-point readmission gap is far smaller than the width of either interval. Death and ICU counts are too small to compare. Charges and length of stay are descriptive only and are not adjusted for DRG mix or age. The slightly higher charges and condition count in Cluster 1 are consistent with its older age profile and do not show a distinct risk group.

Fit provenance. PCA and scalers were fit on complete records within the hospitalized 45–64 cohort, not on the full 100,000-patient table. PCA reduced dimensionality only modestly (4 of 5 clinical features and 11 of 15 diagnosis indicators retained), so K-means is working on nearly the full feature space. Each PCA block was rescaled to unit total variance, which gives each clinical feature roughly three times the weight of each diagnosis indicator. That is a modeling choice and could change the clusters.

Bottom line. This analysis does not support using these segments for care-coordination targeting. The more defensible takeaways are descriptive: overall cohort rates with intervals (Section 4), and the observation that age and blood pressure, not diagnosis profile, account for most of the cluster separation.

Interpretation and next steps

- **Care-coordination hypothesis:** The clusters do not isolate a higher-risk group, so segment-based targeting is not supported by this analysis. A more useful next step would be to adjust readmission for age and case mix, or to test whether any combination of diagnoses predicts readmission directly.
- **Operational monitoring:** Event denominators, charges, and length of stay should be tracked as separate measures. Any comparison across time periods or providers would require case-mix adjustment and consistent eligibility definitions.
- **Evidence limits:** Because the data are synthetic, the generation process may have created or erased associations. For example, diagnosis flags appear to be independent of blood pressure. Cluster separation is weak, and a forced segmentation is not necessarily clinically meaningful.
- **Cohort definition:** The age in the patient table may not match the patient's age at admission. This should be confirmed before relying on the 45–64 boundary.
- **Validation needed:** Future work should test how sensitive the results are to diagnosis weighting, PCA thresholds, block weighting, missing data, and the cohort definition. The segments should also be evaluated for stability on independent or temporally separated data.
- **Not established:** This analysis does not show disease progression over time, prevented admissions, treatment benefits, causal effects, real-world return on investment, or population-wide mortality and readmission forecasts.